Practice: transformation pipelines

Exercise 1. Evaluate map (+3) [1,4,7].
Solution: [4,7,10]. map transforms every element without selecting a subset.

Exercise 2. Evaluate filter (>3) [2,5,1,8].
Solution: [5,8]. filter removes elements whose predicate is False and preserves the order of the survivors.

Exercise 3. Double the even numbers in [1,2,3,4].

```
map (*2) (filter even [1,2,3,4]) == [4,8]
```

The filter runs before map in this expression. Reversing the operations changes the meaning: filtering even after doubling would keep all four doubled inputs.

Exercise 4. Compose length and filter even.

```
countEven = length . filter even
```

countEven [1,2,4,5] returns 2. Composition passes the filtered list to length. The expression is equivalent to countEven xs = length (filter even xs).

Original educational material / CC BY 4.0 / Bilal Amin

Practice: folds and list reconstruction

Exercise 5. Express a sum using foldr.

```
sumFold = foldr (+) 0
```

For [3,4,5], this expands to 3 + (4 + (5 + 0)) and returns 12.

Exercise 6. Rebuild a list using a right fold.

```
foldr (:) [] [1,2,3] == [1,2,3]
```

The list constructor and the empty list preserve the original list's structure and order.

Exercise 7. Reverse a finite list using foldl.

```
reverseFold = foldl (\acc x -> x : acc) []
```

On [1,2,3], accumulators are [], [1], [2,1], and [3,2,1]. Prepending each next element reverses the order.

Exercise 8. Evaluate foldr (-) 0 [4,2] and foldl (-) 0 [4,2].
Solution: foldr gives 4 - (2 - 0) = 2. foldl gives (0 - 4) - 2 = -6.
Subtraction is not associative, so fold direction matters.

Revision check: choose a fold's starting value based on the intended empty-list behavior, not only on a nonempty example.

Original educational material / CC BY 4.0 / Bilal Amin